TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI
KHOA CÔNG NGHỆ THÔNG TIN

BÀI BÁO CÁO BÀI TẬP LỚN

MÔN HỌC: XỬ LÝ NGÔN NGỮ TỰ NHIÊN

ĐỀ TÀI: HỆ THỐNG HỎI ĐÁP TIẾNG VIỆT SỬ DỤNG KIẾN TRÚC RETRIEVER–READER VỚI BM25 VÀ CÁC MÔ HÌNH TRANSFORMER

Họ và Tên: Nguyễn Tất Mạnh

MSSV: 0210768

Lớp: 68CS2

Giảng viên hướng dẫn: ThS Nguyễn Đình Quý

Hà Nội, Ngày 2 Tháng 4 Năm 2026

Mục lục

MỞ ĐẦU **3**
1. Lý do chọn đề tài . . . 3
2. Mục tiêu đề tài . . . 3
3. Phạm vi nghiên cứu . . . 3
4. Cấu trúc báo cáo . . . 3

1 CƠ SỞ LÝ THUYẾT **4**
1.1 Tổng quan về bài toán hỏi đáp . . . 4
1.2 Đặc thù tiếng Việt trong bài toán NLP . . . 4
1.3 Tiền xử lý văn bản và tokenization . . . 4
1.4 Mô hình ngôn ngữ và quy tắc chuỗi xác suất . . . 4
1.5 Transformer và cơ chế self-attention . . . 5
1.6 BERT, XLM-RoBERTa và fine-tuning cho QA . . . 5
1.7 Information Retrieval và BM25 . . . 6
1.8 Mô hình ngôn ngữ lớn và QA sinh câu trả lời . . . 6
1.9 Các chỉ số đánh giá . . . 6

2 DỮ LIỆU VÀ BÀI TOÁN **7**
2.1 Phát biểu bài toán . . . 7
2.2 Bộ dữ liệu sử dụng . . . 7
2.3 Đặc điểm của dữ liệu . . . 7
2.4 Những khó khăn của bài toán . . . 7

3 THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG **8**
3.1 Kiến trúc tổng thể . . . 8
3.2 Thành phần retriever . . . 8
3.3 Reader extractive . . . 9
3.4 Reader generative . . . 9
3.5 Cơ chế chấm điểm và chọn đáp án . . . 9

4 THỰC NGHIỆM, KẾT QUẢ VÀ PHÂN TÍCH LỖI **9**
4.1 Môi trường cài đặt . . . 9
4.2 Thiết lập thực nghiệm . . . 10
4.3 Kết quả định lượng . . . 10
4.3.1 Nhận xét chung . . . 10
4.4 Diễn biến quá trình huấn luyện . . . 10
4.5 Phân tích định tính qua các trường hợp điển hình . . . 11
4.5.1 Trường hợp hai mô hình cùng trả lời đúng nhưng khác cách diễn đạt . . . 11
4.5.2 Trường hợp cả hai mô hình cùng trả lời sai do context và truy hồi . . . 12
4.5.3 Trường hợp cả hai mô hình đúng ý nhưng bị chấm khắt khe theo nhãn . . . 13
4.6 Tổng hợp nguyên nhân gây lỗi . . . 15
4.6.1 Nguyên nhân đến từ dataset . . . 15
4.6.2 Nguyên nhân đến từ hệ thống . . . 15

4.7 Thảo luận 16

5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 16

5.1 Kết luận 16

5.2 Hướng phát triển 16

5.2.1 Cải thiện retriever 16

5.2.2 Cải thiện extractive reader 17

5.2.3 Cải thiện generative reader 17

5.2.4 Cải thiện đánh giá 17

Kết luận chung 17

MỞ ĐẦU

1. Lý do chọn đề tài

Xử lý ngôn ngữ tự nhiên là một trong những hướng nghiên cứu quan trọng của trí tuệ nhân tạo hiện đại. Trong nhiều năm gần đây, các hệ thống hỏi đáp tự động đã trở thành một ứng dụng tiêu biểu của NLP nhờ khả năng tiếp nhận câu hỏi bằng ngôn ngữ tự nhiên và trả về câu trả lời phù hợp. Đối với tiếng Việt, bài toán này có ý nghĩa thực tiễn cao do nhu cầu xây dựng các hệ thống hỗ trợ tra cứu thông tin, trợ lý học tập và hỗ trợ người dùng ngày càng tăng.

Tuy nhiên, việc xây dựng một hệ thống hỏi đáp tiếng Việt vẫn còn nhiều thách thức. Tiếng Việt có đặc thù về tách từ, chuẩn hóa văn bản, ngữ cảnh dài và tên riêng đa âm tiết. Ngoài ra, dữ liệu gán nhãn cho tiếng Việt chưa thật sự phong phú và đồng nhất như tiếng Anh. Vì vậy, việc nghiên cứu và xây dựng một hệ thống hỏi đáp tiếng Việt dựa trên các mô hình hiện đại là cần thiết cả về mặt học thuật lẫn ứng dụng.

Từ những lý do trên, đề tài "Hệ thống hỏi đáp tiếng Việt sử dụng kiến trúc Retriever–Reader với BM25 và các mô hình Transformer" được lựa chọn nhằm khảo sát, cài đặt và đánh giá một pipeline hỏi đáp hoàn chỉnh trên dữ liệu tiếng Việt.

2. Mục tiêu đề tài

Mục tiêu của đề tài bao gồm:

- Xây dựng hệ thống hỏi đáp tiếng Việt hoàn chỉnh theo kiến trúc Retriever–Reader.
- Tích hợp bộ truy hồi văn bản dựa trên BM25 để tìm ngữ cảnh liên quan đến câu hỏi.
- Xây dựng và đánh giá hai nhánh reader:
  - Reader extractive dựa trên XLM-RoBERTa fine-tune cho question answering.
  - Reader generative dựa trên Qwen2.5 để sinh câu trả lời.
- So sánh hai hướng tiếp cận extractive và generative bằng các chỉ số Exact Match, F1 và latency.
- Phân tích các lỗi điển hình, chỉ ra nguyên nhân từ phía dữ liệu và từ phía kiến trúc hệ thống.

3. Phạm vi nghiên cứu

Đề tài tập trung vào bài toán hỏi đáp tiếng Việt dựa trên context trong bộ dữ liệu cho sẵn. Hệ thống được xây dựng theo hướng open-domain ở mức truy hồi ngữ cảnh bằng BM25, sau đó reader thực hiện đọc hiểu hoặc sinh câu trả lời. Trong phạm vi của bài tập lớn, hệ thống chưa triển khai dense retrieval hoàn chỉnh hoặc hybrid retrieval, và phần đánh giá chủ yếu tập trung vào EM, F1, latency, cùng với phân tích lỗi định tính thông qua các ví dụ điển hình.

4. Cấu trúc báo cáo

Báo cáo gồm 5 chương chính. Chương 1 trình bày cơ sở lý thuyết liên quan đến bài toán hỏi đáp tiếng Việt. Chương 2 mô tả dữ liệu, bài toán và các thách thức. Chương 3 trình bày thiết kế hệ thống theo kiến trúc Retriever–Reader. Chương 4 mô tả quá trình thực nghiệm, kết quả đánh giá và phân tích lỗi dựa trên số liệu cùng hình ảnh giao diện. Chương 5 đưa ra kết luận và hướng phát triển.

CHƯƠNG 1 CƠ SỞ LÝ THUYẾT

1.1 Tổng quan về bài toán hỏi đáp

Hệ thống hỏi đáp tự động (Question Answering - QA) là một lớp bài toán trong NLP, trong đó hệ thống nhận đầu vào là một câu hỏi bằng ngôn ngữ tự nhiên và sinh ra câu trả lời tương ứng. Khác với các hệ thống tìm kiếm thông tin truyền thống chỉ trả về danh sách tài liệu liên quan, hệ thống QA hướng đến việc trả về một câu trả lời trực tiếp, ngắn gọn và hữu ích hơn đối với người dùng.

Dựa trên cách thức sinh câu trả lời, hệ thống QA có thể chia thành hai nhóm chính:

- **Extractive QA**: mô hình trích xuất trực tiếp đoạn văn bản hoặc span đáp án từ ngữ cảnh.
- **Generative QA**: mô hình sinh ra câu trả lời mới dựa trên ngữ cảnh, có thể diễn đạt lại nội dung.

Trong đề tài này, hệ thống được xây dựng theo kiến trúc Retriever–Reader. Ở bước đầu, retriever có nhiệm vụ truy hồi các đoạn ngữ cảnh liên quan nhất đến câu hỏi. Ở bước thứ hai, reader xử lý các ngữ cảnh này để tạo ra câu trả lời. Kiến trúc này phù hợp với bài toán hỏi đáp mở miền, nơi hệ thống phải chọn ra thông tin quan trọng trong một tập văn bản lớn trước khi trả lời.

1.2 Đặc thù tiếng Việt trong bài toán NLP

Tiếng Việt có nhiều đặc điểm làm cho việc xử lý ngôn ngữ tự nhiên trở nên khó khăn hơn so với một số ngôn ngữ khác. Trước hết, dấu cách trong tiếng Việt phân tách âm tiết chứ không luôn luôn tương ứng với đơn vị từ. Ví dụ, các cụm "đại học" hoặc "Hà Nội" gồm nhiều âm tiết nhưng lại tạo thành một đơn vị nghĩa thống nhất. Vì vậy, tách từ là một bước quan trọng trong NLP tiếng Việt.

Ngoài ra, tiếng Việt có hệ thống dấu thanh ảnh hưởng trực tiếp đến nghĩa từ, đồng thời tên riêng đa âm tiết xuất hiện nhiều trong văn bản. Điều này khiến bài toán chuẩn hóa văn bản, nhận diện thực thể và truy hồi theo từ khóa cần được xử lý cẩn thận. Nếu tiền xử lý hoặc tokenization không phù hợp, hiệu quả của toàn bộ hệ thống QA sẽ bị giảm đáng kể.

1.3 Tiền xử lý văn bản và tokenization

Tiền xử lý là bước nền tảng trong mọi hệ thống NLP. Đối với hệ thống hỏi đáp tiếng Việt, các thao tác quan trọng bao gồm chuẩn hóa văn bản, tách từ hoặc tokenization, loại bỏ nhiễu và chuyển văn bản thô về dạng mà mô hình có thể tiếp nhận.

Trong các hệ thống NLP hiện đại, tokenization thường được thực hiện ở mức *subword* thay vì chỉ ở mức từ. Các phương pháp như BPE, WordPiece và SentencePiece cho phép chia từ thành các đơn vị nhỏ hơn, từ đó giải quyết tốt hơn bài toán từ ngoài từ vựng và hỗ trợ tốt cho các mô hình pre-trained. Đây cũng là cơ sở để các mô hình như XLM-RoBERTa xử lý dữ liệu đa ngôn ngữ một cách ổn định hơn.

1.4 Mô hình ngôn ngữ và quy tắc chuỗi xác suất

Về mặt lý thuyết, nhiều mô hình NLP được xây dựng dựa trên bài toán mô hình hóa ngôn ngữ. Với một chuỗi từ $w_1, w_2, \ldots, w_n$, xác suất của toàn bộ chuỗi có thể được biểu diễn theo quy tắc chuỗi như

sau:

$$P(w_1, w_2, \ldots, w_n) = \prod_{i=1}^{n} P(w_i \mid w_1, w_2, \ldots, w_{i-1}) \tag{1.1}$$

Công thức trên cho thấy việc hiểu hoặc sinh ngôn ngữ phụ thuộc chặt chẽ vào ngữ cảnh trước đó. Các mô hình ngôn ngữ cổ điển như n-gram xấp xỉ ngữ cảnh dài bằng một cửa sổ hữu hạn:

$$P(w_i \mid w_1, \ldots, w_{i-1}) \approx P(w_i \mid w_{i-n+1}, \ldots, w_{i-1}) \tag{1.2}$$

Các mô hình Transformer và LLM hiện đại thay thế giả định n-gram bằng cơ chế self-attention, cho phép khai thác ngữ cảnh dài tốt hơn.

1.5 Transformer và cơ chế self-attention

Transformer là một bước đột phá lớn trong NLP hiện đại. Không giống RNN hay LSTM xử lý tuần tự theo thời gian, Transformer cho phép xử lý song song toàn bộ chuỗi đầu vào. Thành phần quan trọng nhất của kiến trúc này là self-attention.

Với đầu vào được biểu diễn dưới dạng vector x_i, cơ chế attention sử dụng ba phép chiếu tuyến tính:

$$q_i = W_Q x_i, \quad k_i = W_K x_i, \quad v_i = W_V x_i \tag{1.3}$$

Điểm attention giữa vị trí i và j được tính:

$$e_{ij} = q_i^T k_j \tag{1.4}$$

Sau khi chuẩn hóa bằng softmax:

$$\alpha_{ij} = \frac{\exp(e_{ij}/\sqrt{d_k})}{\sum_{l=1}^{n} \exp(e_{il}/\sqrt{d_k})} \tag{1.5}$$

Biểu diễn mới tại vị trí i:

$$z_i = \sum_{j=1}^{n} \alpha_{ij} v_j \tag{1.6}$$

Nhờ self-attention, mô hình có thể học được các mối quan hệ xa trong câu và tập trung vào các token quan trọng khi trả lời câu hỏi.

1.6 BERT, XLM-RoBERTa và fine-tuning cho QA

BERT là mô hình encoder-only nổi tiếng dựa trên Transformer, được huấn luyện trước trên lượng dữ liệu lớn và sau đó fine-tune cho từng tác vụ cụ thể. Các mô hình cùng họ như RoBERTa, XLM-RoBERTa và PhoBERT đã chứng minh hiệu quả rất cao trên nhiều bài toán NLP.

Đối với question answering dạng extractive, đầu vào của mô hình thường là cặp *question + context*. Mục tiêu của mô hình là xác định vị trí bắt đầu và kết thúc của đáp án trong context. Trong đề tài này, nhánh extractive sử dụng XLM-RoBERTa đã được fine-tune cho bài toán QA tiếng Việt.

Quá trình fine-tuning mô hình được tối ưu thông qua hàm mất mát cross-entropy. Với xác suất dự đoán p và nhãn đúng y, hàm mất mát nhị phân có thể viết:

$$L = -y \log(p) - (1 - y) \log(1 - p) \tag{1.7}$$

1.7 Information Retrieval và BM25

Để hệ thống hoạt động theo hướng open-domain, việc truy hồi các context liên quan là bước bắt buộc. Trong đề tài này, retriever được xây dựng dựa trên BM25, một mô hình xếp hạng văn bản cổ điển nhưng hiệu quả trong Information Retrieval.

BM25 có thể được biểu diễn như sau:

$$BM25(D,Q) = \sum_{i=1}^{n} IDF(q_i) \cdot \frac{f(q_i,D)\cdot(k_1+1)}{f(q_i,D)+k_1\cdot\left(1-b+b\cdot\frac{|D|}{avgdl}\right)} \tag{1.8}$$

Trong đó:

- $f(q_i, D)$ là số lần từ khóa q_i xuất hiện trong tài liệu D;
- $|D|$ là độ dài tài liệu;
- $avgdl$ là độ dài trung bình của tập tài liệu;
- k_1 và b là các siêu tham số;
- $IDF(q_i)$ phản ánh mức độ quan trọng của từ khóa trong toàn bộ tập tài liệu.

Ý nghĩa của BM25 là kết hợp giữa mức độ khớp từ khóa, độ hiếm của từ và độ dài tài liệu. Tuy nhiên, nhược điểm lớn của BM25 là phụ thuộc mạnh vào trùng khớp từ bề mặt, do đó có thể bỏ sót những tài liệu đúng ý nhưng dùng cách diễn đạt khác.

1.8 Mô hình ngôn ngữ lớn và QA sinh câu trả lời

Khác với extractive QA, generative QA không bắt buộc câu trả lời phải là một span nằm trong context. Thay vào đó, mô hình sử dụng ngữ cảnh và câu hỏi để sinh ra câu trả lời theo từng token. Các mô hình dạng này thường thuộc lớp decoder-only Transformer.

Ưu điểm của generative QA là khả năng diễn đạt câu trả lời tự nhiên hơn, dễ hiểu hơn đối với người dùng. Tuy nhiên, nhược điểm là mô hình có thể sinh ra câu trả lời không trùng hoàn toàn với nhãn chuẩn, dẫn đến bị giảm điểm khi đánh giá bằng Exact Match và F1. Ngoài ra, nhánh generative thường có thời gian suy luận lớn hơn đáng kể so với extractive.

1.9 Các chỉ số đánh giá

Đề tài sử dụng ba chỉ số chính để đánh giá chất lượng hệ thống:

- **Exact Match (EM):** đo tỷ lệ câu trả lời dự đoán trùng hoàn toàn với đáp án chuẩn sau khi chuẩn hóa.
- **F1 Score:** đo mức độ chồng lắp token giữa câu trả lời dự đoán và đáp án chuẩn.
- **Latency:** đo thời gian phản hồi trung bình của hệ thống.

Ngoài ra, đối với nhánh extractive, báo cáo còn theo dõi **Train Loss** và **Validation Loss** theo từng epoch để đánh giá quá trình hội tụ và nguy cơ overfitting.

CHƯƠNG 2 DỮ LIỆU VÀ BÀI TOÁN

2.1 Phát biểu bài toán

Bài toán đặt ra trong đề tài là xây dựng một hệ thống hỏi đáp tiếng Việt có khả năng tiếp nhận câu hỏi của người dùng và trả về câu trả lời tương ứng dựa trên tập ngữ cảnh có sẵn. Hệ thống phải hoạt động theo pipeline hai bước: truy hồi ngữ cảnh liên quan bằng retriever, sau đó dùng reader để trả lời.

Đầu vào của hệ thống là một câu hỏi tiếng Việt. Đầu ra là câu trả lời ở một trong hai dạng:

- một span ngắn được trích xuất từ context (extractive);
- một câu trả lời tự nhiên được sinh ra từ ngữ cảnh (generative).

2.2 Bộ dữ liệu sử dụng

Hệ thống sử dụng bộ dữ liệu hỏi đáp tiếng Việt `ntphuc149/ViSpanExtractQA`. Mỗi mẫu dữ liệu có cấu trúc gồm ba trường chính:

- `context`: đoạn ngữ cảnh;
- `question`: câu hỏi;
- `answer_text`: đáp án chuẩn.

Bộ dữ liệu này phù hợp cho cả hai hướng tiếp cận extractive và generative. Đối với extractive, đáp án thường ngắn và xuất hiện trực tiếp trong context. Đối với generative, mô hình có thể dùng cùng dữ liệu nhưng sinh lại câu trả lời theo ngôn ngữ tự nhiên.

2.3 Đặc điểm của dữ liệu

Qua khảo sát thực tế, bộ dữ liệu có một số đặc điểm đáng chú ý:

- Phần lớn câu trả lời ngắn, thường là tên riêng, địa danh, số lượng hoặc cụm thông tin ngắn.
- Context có độ dài khá đa dạng, từ những đoạn ngắn đến các đoạn dài chứa nhiều thông tin phụ.
- Nhiều câu hỏi có thể trả lời chính xác bằng một span ngắn trong context, do đó rất phù hợp với extractive QA.
- Một số mẫu chứa nhiều thực thể, nhiều con số hoặc cấu trúc dài, gây nhiễu cho cả retriever lẫn reader.

2.4 Những khó khăn của bài toán

Mặc dù dữ liệu phù hợp cho QA, bài toán vẫn đối mặt với một số khó khăn:

- Câu trả lời của mô hình có thể đúng về mặt ngữ nghĩa nhưng không trùng hoàn toàn với đáp án chuẩn, đặc biệt ở nhánh generative.
- Một số context dài chứa nhiều thực thể khiến reader chọn sai span đáp án.

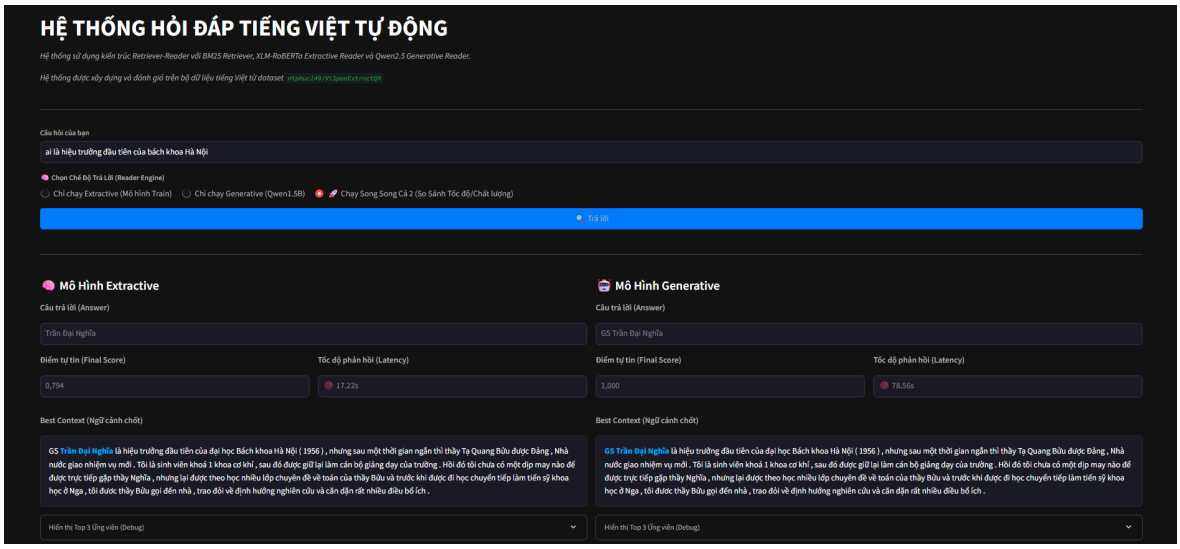
- Retriever BM25 mạnh về khớp từ khóa nhưng có thể bỏ sót các trường hợp câu hỏi và context không trùng từ bề mặt.
- Cách gán nhãn answer_text trong dataset đôi khi rất ngắn, khiến EM bị nghiêm ngặt hơn trải nghiệm thực tế.

CHƯƠNG 3 THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG

3.1 Kiến trúc tổng thể

Hệ thống được xây dựng theo kiến trúc Retriever–Reader. Luồng hoạt động tổng quát gồm các bước:

1. Người dùng nhập câu hỏi tiếng Việt.
2. Retriever sử dụng BM25 để truy hồi các context liên quan nhất.
3. Reader nhận từng cặp *question + context* và sinh ra các đáp án ứng viên.
4. Hệ thống tính điểm, xếp hạng các đáp án và chọn ra câu trả lời cuối cùng.
5. Kết quả được hiển thị qua giao diện web cùng với latency và context tốt nhất.



Hình 3.1: Giao diện tổng quát của hệ thống hỏi đáp tiếng Việt tự động

3.2 Thành phần retriever

Retriever trong hệ thống được xây dựng dựa trên BM25. Khi khởi động, hệ thống duyệt qua toàn bộ tập context, chuẩn bị danh sách văn bản và xây chỉ mục cho bước truy hồi. Khi người dùng đặt câu hỏi, retriever tính điểm BM25 giữa câu hỏi và từng context, sau đó lấy ra top-k context có độ liên quan cao nhất.

Ưu điểm của BM25 trong đề tài là:

- dễ triển khai;
- tốc độ nhanh;

- không cần huấn luyện;
- hiệu quả khá tốt với các câu hỏi có độ khớp từ khóa cao với context.

Tuy nhiên, BM25 cũng có hạn chế. Nếu câu hỏi và context dùng từ vựng khác nhau để diễn đạt cùng một ý, retriever có thể không lấy được context tốt nhất. Đây là một nguyên nhân quan trọng gây lỗi trong hệ thống hiện tại.

3.3 Reader extractive

Nhánh extractive sử dụng mô hình XLM-RoBERTa đã được fine-tune cho bài toán question answering. Với đầu vào là cặp *question + context*, mô hình dự đoán vị trí bắt đầu và vị trí kết thúc của đáp án trong context. Sau đó, span tương ứng được cắt ra làm câu trả lời.

Nhánh này đặc biệt phù hợp với bộ dữ liệu của đề tài, vì phần lớn `answer_text` là những đoạn ngắn xuất hiện trực tiếp trong context. Ngoài ra, nhánh extractive có tốc độ suy luận nhanh, giúp hệ thống phản hồi gần như tức thời.

3.4 Reader generative

Nhánh generative sử dụng mô hình Qwen2.5 để sinh câu trả lời từ câu hỏi và context. Thay vì trích xuất span, mô hình sinh ra từng token theo cơ chế autoregressive. Cách làm này giúp câu trả lời tự nhiên và đầy đủ hơn.

Tuy nhiên, nhánh generative cũng có hai nhược điểm rõ rệt:

- thời gian suy luận lớn;
- dễ bị giảm điểm EM/F1 do diễn đạt lại đáp án thay vì lặp đúng chuỗi `answer_text`.

Vì vậy, trong hệ thống của đề tài, generative reader đóng vai trò là hướng so sánh và mở rộng hơn là nhánh chính để tối ưu chỉ số.

3.5 Cơ chế chấm điểm và chọn đáp án

Sau khi retriever trả về top-k context, reader xử lý từng context và sinh ra danh sách các đáp án ứng viên. Mỗi đáp án có thể được gắn với:

- điểm từ retriever;
- điểm tin cậy của reader;
- điểm tổng hợp cuối cùng.

Hệ thống sau đó sắp xếp các ứng viên theo điểm tổng hợp để chọn ra đáp án tốt nhất. Cách làm này giúp hệ thống tận dụng đồng thời cả độ liên quan của context và độ tin cậy của reader.

CHƯƠNG 4 THỰC NGHIỆM, KẾT QUẢ VÀ PHÂN TÍCH LỖI

4.1 Môi trường cài đặt

Hệ thống được xây dựng bằng Python với các thư viện chính như:

- `transformers` cho các mô hình Transformer và LLM;
- `FastAPI` cho backend;
- `Streamlit` cho giao diện người dùng;
- thư viện BM25 cho bước truy hồi;
- các thư viện hỗ trợ huấn luyện và đánh giá khác trong hệ sinh thái PyTorch/Hugging Face.

4.2 Thiết lập thực nghiệm

Các thực nghiệm được tiến hành trên hai nhánh mô hình:

- **Extractive**: XLM-RoBERTa fine-tune cho QA;
- **Generative**: Qwen2.5 sinh câu trả lời từ context.

Việc đánh giá được thực hiện trên hai quy mô:

- 500 mẫu;
- 5000 mẫu.

Ba chỉ số được sử dụng là Exact Match, F1 và Avg Latency.

4.3 Kết quả định lượng

Kết quả đánh giá của hệ thống được trình bày trong Bảng 4.1.

Bảng 4.1: Kết quả đánh giá hệ thống

Mô hình	**Số mẫu**	**EM**	**F1**	**Latency**
Extractive	500	0.5560	0.7763	18.40 ms
Extractive	5000	0.4634	0.6875	19.72 ms
Generative	500	0.2820	0.6058	889.76 ms
Generative	5000	0.1844	0.4565	891.44 ms

4.3.1 Nhận xét chung

Kết quả cho thấy nhánh extractive đạt hiệu quả tốt hơn rõ rệt so với nhánh generative trên cả hai chỉ số EM và F1. Cụ thể, trên 500 mẫu, mô hình extractive đạt EM = 55.60% và F1 = 77.63%. Trên 5000 mẫu, mô hình vẫn duy trì EM = 46.34% và F1 = 68.75%, cho thấy khả năng tổng quát hóa tương đối ổn định.

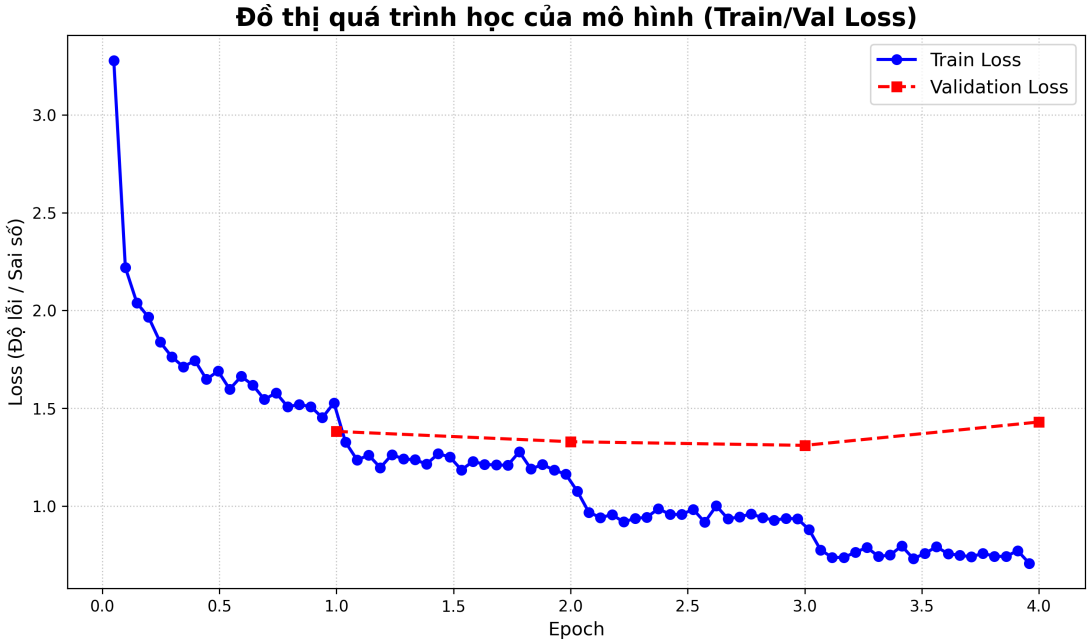
Trong khi đó, nhánh generative cho câu trả lời tự nhiên hơn nhưng bị giảm điểm số do không khớp hoàn toàn với nhãn chuẩn. Đồng thời, latency của generative lớn hơn rất nhiều so với extractive, khiến nó chưa phù hợp cho các kịch bản cần phản hồi nhanh.

4.4 Diễn biến quá trình huấn luyện

Đối với nhánh extractive, quá trình huấn luyện được theo dõi bằng *Train Loss* và *Validation Loss*. Bảng 4.2 trình bày các giá trị loss theo từng epoch.

Bảng 4.2: Train Loss và Validation Loss theo từng epoch

Epoch	Train Loss	Validation Loss
0	3.2785	N/A
1	1.5974	1.3817
2	1.1849	1.3290
3	0.9832	1.3103
4	0.7578	1.4300



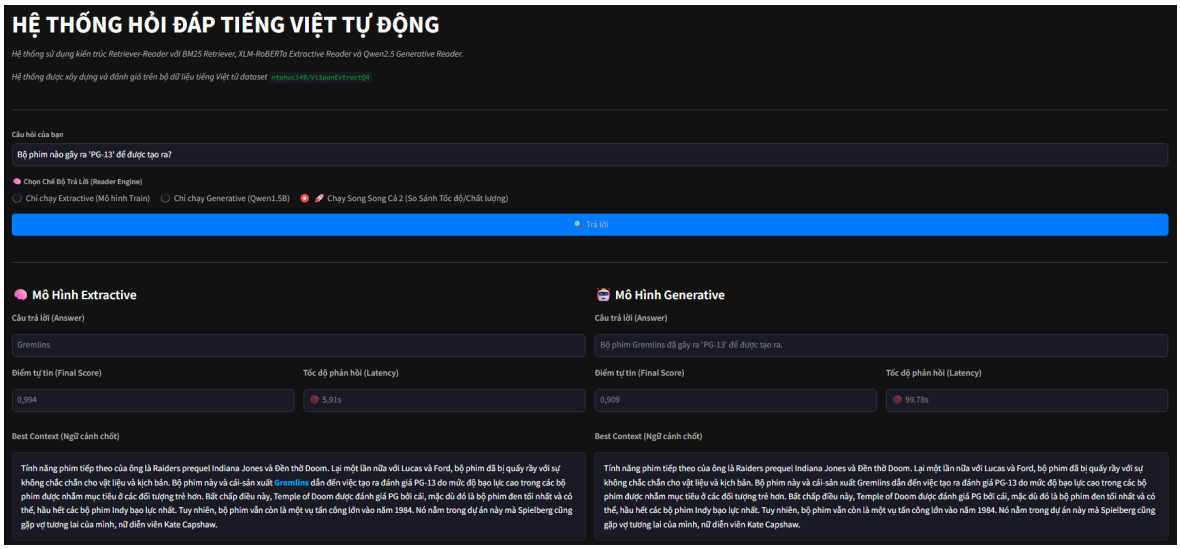
Hình 4.1: Đồ thị quá trình học của mô hình extractive theo Train Loss và Validation Loss

Từ Bảng 4.2 và Hình 4.1, có thể thấy train loss giảm đều qua các epoch, chứng tỏ mô hình học được các mẫu trong dữ liệu huấn luyện. Validation loss giảm từ epoch 1 đến epoch 3, nhưng tăng trở lại ở epoch 4. Điều này cho thấy mô hình bắt đầu có dấu hiệu *overfitting nhẹ* sau epoch 3. Do đó, checkpoint tốt nhất về mặt tổng quát hóa nên được chọn quanh epoch 3 hoặc kết hợp với cơ chế early stopping.

4.5 Phân tích định tính qua các trường hợp điển hình

4.5.1 Trường hợp hai mô hình cùng trả lời đúng nhưng khác cách diễn đạt

Hình 4.2 minh họa một ví dụ mà cả extractive và generative đều trả lời đúng ý câu hỏi "Bộ phim nào gây ra 'PG-13' để được tạo ra?". Trong trường hợp này, extractive trả lời ngắn gọn bằng span "Gremlins", trong khi generative sinh câu đầy đủ hơn: "Bộ phim Gremlins đã gây ra 'PG-13' để được tạo ra.".

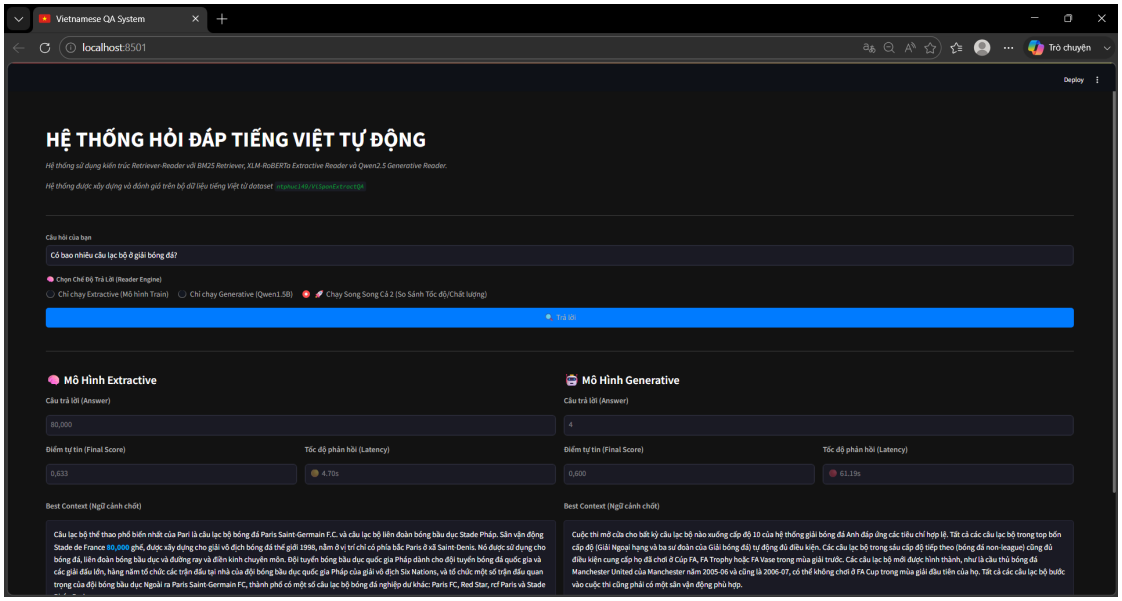


Hình 4.2: Ví dụ cả extractive và generative đều đúng, nhưng generative diễn đạt lại tự nhiên hơn

Trường hợp này cho thấy EM và F1 chưa phản ánh đầy đủ chất lượng ngữ nghĩa của generative. Về phía người dùng, cả hai câu trả lời đều đúng. Tuy nhiên, câu trả lời của generative dài hơn nên dễ bị giảm EM hơn so với extractive.

4.5.2 Trường hợp cả hai mô hình cùng trả lời sai do context và truy hồi

Hình 4.3 và Hình 4.4 minh họa một ví dụ điển hình của lỗi hệ thống với câu hỏi "Có bao nhiêu câu lạc bộ ở giải bóng đá?". Nhãn trong dataset là "36 câu lạc bộ trong giải bóng đá", nhưng hệ thống trả về "80,000" ở nhánh extractive và "4" ở nhánh generative.



Hình 4.3: Ví dụ cả extractive và generative cùng trả lời sai

Hình 4.4: Nhãn chuẩn trong dataset cho câu hỏi về số lượng câu lạc bộ

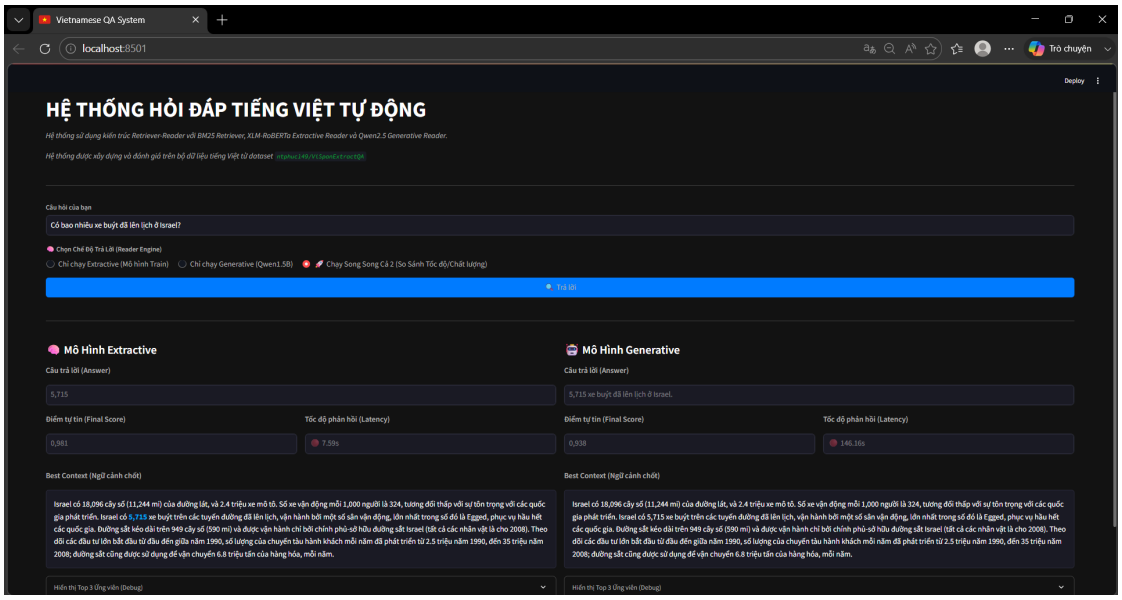
Nguyên nhân của lỗi này có thể phân tích theo hai phía:

- **Do retriever**: BM25 ưu tiên trùng khớp từ khóa bề mặt. Với các câu hỏi có nhiều từ chung như "câu lạc bộ", "giải bóng đá", retriever có thể kéo về một context không thật sự đúng trọng tâm.
- **Do reader**: khi context đã sai hoặc chỉ gần đúng, reader vẫn buộc phải sinh hoặc trích một đáp án từ ngữ cảnh đó, dẫn đến kết quả sai.

Đây là nhóm lỗi phản ánh giới hạn hiện tại của hệ thống, đặc biệt ở thành phần retriever.

4.5.3 Trường hợp cả hai mô hình đúng ý nhưng bị chấm khắt khe theo nhãn

Hình 4.5 và Hình 4.6 cho thấy một câu hỏi "Có bao nhiêu xe buýt đã lên lịch ở Israel?". Extractive trả lời "5,715" còn generative trả lời "5,715 xe buýt đã lên lịch ở Israel.". Trong trường hợp này, cả hai câu trả lời đều đúng về ngữ nghĩa; extractive gần với nhãn hơn, còn generative diễn đạt tự nhiên hơn.

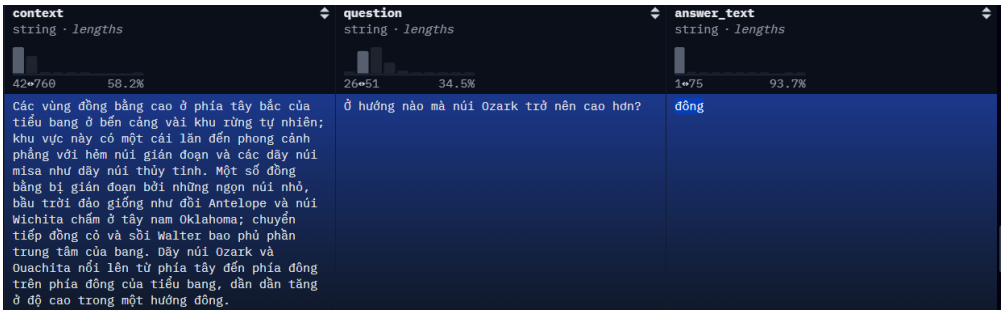


Hình 4.5: Ví dụ cả hai mô hình đều đúng ý; generative sinh câu đầy đủ hơn

context	question	answer_text
string · lengths	string · lengths	string · lengths
42↔760 58.2%	26↔51 34.5%	1↔75 93.7%
Israel có 18,096 cây số (11,244 mi) của đường lát, và 2.4 triệu xe mô tô. Số xe vận động mỗi 1,000 người là 324, tương đối thấp với sự tôn trọng với các quốc gia phát triển. Israel có 5,715 xe buýt trên các tuyến đường đã lên lịch, vận hành bởi một số sản vận động, lớn nhất trong số đó là Egged, phục vụ hầu hết các quốc gia. Đường sắt kéo dài trên 949 cây số (590 mi) và được vận hành chỉ bởi chính phủ-sở hữu đường sắt Israel (tất cả các nhân vật là cho 2008). Theo dõi các đầu tư lớn bắt đầu từ đầu đến giữa năm 1990, số lượng của chuyến tàu hành khách mỗi năm đã phát triển từ 2.5 triệu năm 1990, đến 35 triệu năm 2008; đường sắt cũng được sử dụng để vận chuyển 6.8 triệu tấn của hàng hóa, mỗi năm.	Có bao nhiêu xe buýt đã lên lịch ở Israel?	5,715

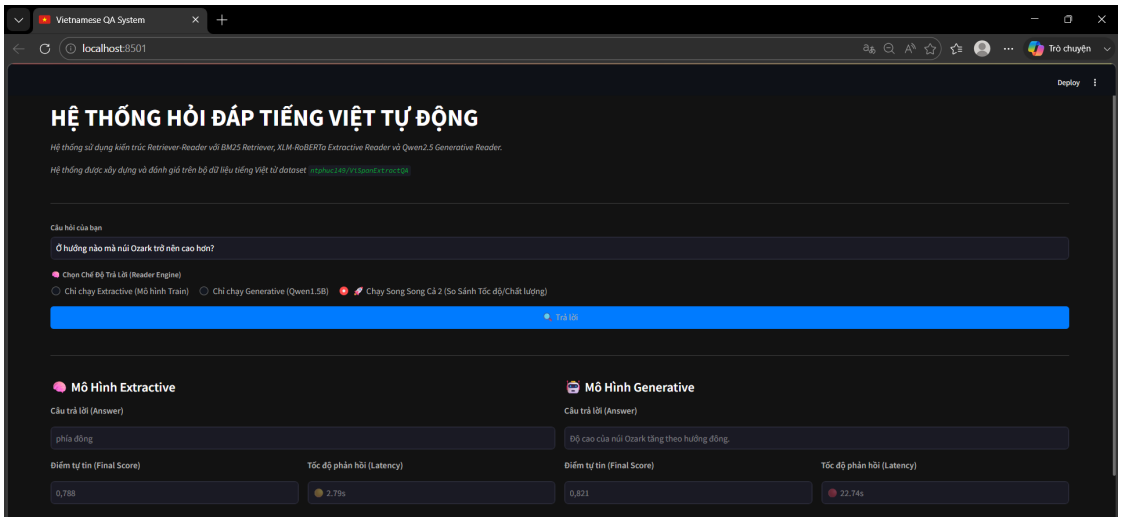
Hình 4.6: Nhãn chuẩn trong dataset cho câu hỏi về số xe buýt ở Israel

Một ví dụ khác là câu hỏi "Ở hướng nào mà núi Ozark trở nên cao hơn?". Dataset gán nhãn ngắn là "đông", trong khi extractive trả lời "phía đông" và generative trả lời "Độ cao của núi Ozark tăng theo hướng đông.". Cả hai câu trả lời thực chất đều đúng nghĩa, nhưng extractive có thể bị giảm EM do thừa từ, còn generative gần như chắc chắn không trùng hoàn toàn với answer_text.



context	question	answer_text
string · lengths	string · lengths	string · lengths
42↔760 58.2%	26↔51 34.5%	1↔75 93.7%
Các vùng đồng bằng cao ở phía tây bắc của tiểu bang ở bến cảng vài khu rừng tự nhiên; khu vực này có một cái lăn đến phong cảnh phẳng với hẻm núi gián đoạn và các dãy núi mỉsa như dãy núi thủy tinh. Một số đồng bằng bị gián đoạn bởi những ngọn núi nhỏ, bầu trời đảo giống như đồi Antelope và núi Wichita chấm ở tây nam Oklahoma; chuyển tiếp đồng cỏ và sồi Walter bao phủ phần trung tâm của bang. Dãy núi Ozark và Ouachita nổi lên từ phía tây đến phía đông trên phía đông của tiểu bang, dần dần tăng ở độ cao trong một hướng đông.	Ở hướng nào mà núi Ozark trở nên cao hơn?	đông

Hình 4.7: Nhãn chuẩn trong dataset cho câu hỏi về hướng cao dần của núi Ozark



Hình 4.8: Ví dụ extractive và generative đều đúng ý nhưng khác bề mặt nhãn

4.6 Tổng hợp nguyên nhân gây lỗi

4.6.1 Nguyên nhân đến từ dataset

Từ các ví dụ trên, có thể thấy một phần lỗi không hoàn toàn đến từ mô hình mà đến từ đặc điểm của dataset:

- **Nhãn quá ngắn hoặc quá cứng**: nhiều answer_text chỉ là một từ hoặc một cụm rất ngắn, trong khi generative có xu hướng trả lời đầy đủ hơn.
- **Không phản ánh đầy đủ các biến thể đúng**: ví dụ "đông" và "phía đông" cùng đúng, nhưng EM chỉ chấp nhận một dạng.
- **Ngữ cảnh dài và nhiều nhiễu**: có những context chứa nhiều số liệu, nhiều thực thể hoặc nhiều câu gần nghĩa, làm cho cả retriever và reader dễ nhầm lẫn.

Như vậy, có những trường hợp mô hình thực tế trả lời hợp lý nhưng vẫn bị tính sai hoặc mất điểm khi đánh giá bằng EM/F1.

4.6.2 Nguyên nhân đến từ hệ thống

Ngoài yếu tố dữ liệu, hệ thống hiện tại cũng chưa được cải tiến đến mức tối đa:

- **Retriever vẫn là BM25 thuần**: hệ thống chưa có dense retrieval hoặc hybrid retrieval để xử lý lexical mismatch.
- **Reader extractive chưa có hậu xử lý span mạnh**: các trường hợp "đông" so với "phía đông" cho thấy span boundary vẫn có thể tối ưu thêm.
- **Reader generative chưa được ràng buộc chặt với answer format**: generative ưu tiên tự nhiên hơn là khớp đúng answer_text.
- **Giới hạn của mô hình đọc ngữ cảnh dài**: với các context dài và nhiều thực thể, cả extractive lẫn generative đều có thể bị nhiễu.

4.7 Thảo luận

Từ toàn bộ kết quả, có thể rút ra ba kết luận chính.

Thứ nhất, nhánh extractive là thành phần mạnh nhất của hệ thống hiện tại. Nó đạt EM và F1 cao hơn rõ rệt, đồng thời có latency rất thấp, phù hợp cho triển khai thực tế.

Thứ hai, nhánh generative có giá trị riêng ở khả năng diễn đạt tự nhiên, nhưng không phù hợp để tối ưu theo EM/F1. Khi dataset sử dụng answer_text ngắn, generative gần như luôn bị bất lợi so với extractive.

Thứ ba, kết quả thực nghiệm cho thấy lỗi của hệ thống đến từ cả hai phía: **dataset** và **kiến trúc hiện tại**. Không nên quy toàn bộ lỗi cho mô hình. Có những lỗi xuất phát từ cách gán nhãn quá cứng, nhưng cũng có những lỗi xuất phát từ retriever và từ việc hệ thống chưa tận dụng các kỹ thuật nâng cao như hybrid retrieval, better span post-processing hay answer normalization.

CHƯƠNG 5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Trong phạm vi bài tập lớn môn Xử lý ngôn ngữ tự nhiên, đề tài đã xây dựng thành công một hệ thống hỏi đáp tiếng Việt theo kiến trúc Retriever–Reader với BM25 và các mô hình Transformer. Hệ thống có đầy đủ các thành phần chính gồm retriever, extractive reader, generative reader, backend API và giao diện web phục vụ tương tác thực tế.

Về mặt thực nghiệm, kết quả cho thấy nhánh extractive là thành phần mạnh nhất của hệ thống. Mô hình đạt EM = 55.60%, F1 = 77.63% trên 500 mẫu và EM = 46.34%, F1 = 68.75% trên 5000 mẫu, với latency dưới 20 ms. Đây là mức kết quả tốt và phù hợp với một bài tập lớn ở bậc đại học. Trong khi đó, nhánh generative cho câu trả lời tự nhiên hơn nhưng bị hạn chế bởi latency cao và mức độ tương thích thấp hơn với chỉ số EM/F1.

Bên cạnh kết quả định lượng, các ví dụ định tính trong báo cáo cho thấy hệ thống có thể rơi vào ba tình huống khác nhau: cùng đúng nhưng diễn đạt khác nhau, cùng sai do truy hồi/context, và đúng ý nhưng lệch bề mặt so với answer_text. Các trường hợp này giúp khẳng định rằng việc đánh giá một hệ thống QA không nên chỉ nhìn vào điểm số cuối cùng, mà cần kết hợp cả phân tích lỗi và quan sát đầu ra thực tế.

5.2 Hướng phát triển

Mặc dù đã đạt được những kết quả tích cực, hệ thống vẫn còn nhiều hướng phát triển trong tương lai.

5.2.1 Cải thiện retriever

Có thể nâng cấp retriever bằng cách:

- tách từ tiếng Việt tốt hơn trước khi lập chỉ mục;
- kết hợp BM25 với dense retrieval;
- triển khai hybrid retrieval để tận dụng cả khớp từ khóa và khớp ngữ nghĩa.

5.2.2 Cải thiện extractive reader

Có thể tiếp tục tối ưu nhánh extractive bằng:

- thử các backbone phù hợp tiếng Việt hơn như PhoBERT;
- tối ưu thêm hyperparameter;
- cải thiện hậu xử lý span để tăng Exact Match.

5.2.3 Cải thiện generative reader

Có thể phát triển nhánh generative theo hướng:

- thiết kế prompt tốt hơn;
- dùng generative như bước diễn đạt lại sau khi extractive đã tìm được đáp án;
- bổ sung cơ chế kiểm tra câu trả lời có được hỗ trợ bởi context hay không.

5.2.4 Cải thiện đánh giá

Ngoài EM, F1 và latency, các hướng đánh giá tiếp theo có thể bao gồm:

- Recall@k hoặc MRR cho retriever;
- ROUGE/BERTScore cho generative;
- answer normalization để giảm các trường hợp đúng ý nhưng bị chấm sai;
- phân tích lỗi chi tiết hơn theo từng nhóm câu hỏi.

Kết luận chung

Nhìn chung, đề tài là một bước thử nghiệm có hệ thống trong việc kết hợp truy hồi thông tin và đọc hiểu máy cho tiếng Việt, đồng thời cung cấp cơ sở thực nghiệm có giá trị cho các nghiên cứu tiếp theo về tối ưu hóa kiến trúc hỏi đáp, nâng cao khả năng tổng quát hóa và mở rộng tính ứng dụng của các mô hình ngôn ngữ trong bối cảnh tiếng Việt.